Morbius: FFN Capacity Reallocation in Memory-Augmented Language Models via Latent N-gram Tables

Aditya Shankar **Sparsh Bhartia** **Udith Krishna M Nair** **Adithya Kishor**
IIIT Hyderabad
{aditya.shankar, sparsh.bhartia, udithkrishna.nair, adithya.kishor}
@research.iiit.ac.in

Abstract

Engram and Lngram add an n-gram lookup table to a Transformer so that local patterns can be read from memory instead of rebuilt by the network. We asked whether this makes some feed-forward (FFN) capacity redundant. We trained six Llama-style models with ∼370M non-embedding parameters each on 2B tokens, varying whether Lngram v2 memory is present, whether 31.5M FFN parameters move into wider attention, where the FFN is cut, and a rank-1 readout. Moving parameters into attention gave the best model (2.774 against 2.802 for the dense baseline), but helped the dense model about twice as much as the memory model. Memory tied dense at equal total parameters with 27% fewer active parameters per token. These comparisons are single-seed and cross GPU types, so we treat them as provisional. We argue that equal total parameters is the wrong yardstick for memory, since a sparse table can sit in host RAM while dense weights cannot, and plan comparisons at equal GPU-resident parameters, a table-share sweep, an Engram baseline, downstream benchmarks and offload latency.

1 Introduction

A Transformer (Vaswani et al., 2017) has no direct way to look something up. When it needs a fixed phrase or a fact, it rebuilds it through attention and feed-forward computation, often over several layers. An example used in both the Engram and Lngram papers is the phrase "Diana, Princess of Wales": in a standard model, the representation only settles on the specific person around layer 6 (Cheng et al., 2026; Zheng et al., 2026b).

Engram (Cheng et al., 2026) and Lngram (Zheng et al., 2026b,a) replace part of that work with an actual lookup. A table indexed by the last few tokens (Engram) or by codes computed from the hidden state (Lngram) returns a vector that is added to the residual stream. Both papers report that this frees up depth: CKA and LogitLens (nostalgebraist, 2020) profiles of the memory model look like those of a deeper baseline. Neither paper then makes the backbone smaller to take advantage of it. The one attempt, Lngram's 23-layer variant, deletes a whole block. Its average score goes up, but MMLU and PIQA drop, which suggests the deletion also removed computation the memory could not replace.

Our proposal took this as the starting point. If memory takes over local pattern work, and that work lives mostly in the FFNs of the early and middle layers, then some FFN width should be removable without loss, and the parameters can go somewhere more useful. This report covers the first round of experiments. We built a 2×2 design (memory on or off, FFN cut or not) at a fixed total of ∼370M non-embedding parameters, plus a cut-location control and a readout variant. We expected memory to soften the cost of cutting the FFN. It did the opposite: the cut helped the dense model more.

That result, together with memory only tying dense, made us ask whether the memory earns its place at all at this scale. Section 7 argues that our comparisons so far cannot answer this, because they hold total parameters fixed. Only a few table rows are read per token, so the table can in principle be served from system RAM while the GPU holds the rest of the model. The right test is quality at equal GPU-resident parameters, and our plan (Section 8) is built around it.

This report contributes a parameter-matched ablation of FFN-to-attention reallocation with and without memory, a probe of how much each model relies on its memory, an account of where we departed from the proposal, and a reframed question with a plan for the final submission.

2 Related Work

Lookup memory for language models. The common idea is sparse lookup: only a small fraction of the stored parameters is touched per token, so total capacity grows without a matching growth in per-token compute. **Engram** (Cheng et al., 2026) builds its key by hashing the token IDs of the last few tokens. Because the key depends only on the input text, the rows a token needs are known before the forward pass and can be prefetched; with a 100B-parameter table held entirely in host DRAM, the authors measure at most 2.8% lower inference throughput on 4B and 8B dense backbones (H800, 512 sequences). Engram also gives a useful probe: zeroing the memory at inference cut factual tasks to 29–44% of their score, while reading comprehension kept 81–93%. Its weakness is the hash. Unseen n-grams either miss or collide, and nothing about the lookup itself is learned.

Lngram v1 (Zheng et al., 2026b) builds the key from the model's own hidden state instead. A learned router projects the hidden state, takes the sign of each dimension, and groups the bits into short codes; 2-gram and 3-gram combinations of codes index the table. Thresholding is not differentiable, so the router is trained with a surrogate gradient that compares what would have been retrieved under nearby codes. This matters a great deal: in the v2 paper a straight-through estimator stalls at 7.33 validation loss against 2.86 with the surrogate (Zheng et al., 2026a). Lngram v1 is also the source of the effective-depth evidence that motivates us, and of the 23-layer experiment. That model beats the 24-layer MoE baseline on average (0.5207 vs. 0.5146), but on MMLU and PIQA it falls below both the 24-layer Lngram model (MMLU 0.2619 to 0.2300, near chance; PIQA 0.6926 to 0.6740) and the plain baseline (0.2435 and 0.6801). The authors conclude that the memory substitutes for a subset of local pattern modelling, not for Transformer depth in general. Lngram v1 also runs the allocation study most relevant to us. With an MoE backbone, giving memory about 25% of the sparse capacity was best, and both 15% and 35% were worse. Engram reports a similar optimum of roughly 20–25%. Two further results bear on our setup: Lngram's gain over the baseline shrank from 1.41 to 0.38 points when training went from 35B to 140B tokens, and every domain-adaptation method they tried hurt general benchmarks.

Lngram v2 (Zheng et al., 2026a) reworks the readout. A single key-value projection is shared across routes, and the routes are read with grouped-query attention, which decouples the number of routes, the memory dimension and the backbone width. Routes are processed in chunks so that their activations need not all be stored at once. It adds a zero-value "Sink" slot that lets the model say nothing useful was retrieved. Its analysis reports about 8 effective codes per route out of 16. It also finds that the memory's addresses are unstable when it is added to a model whose representations are still changing, especially at the deeper insertion layer, and recommends inserting it after they have settled. We use v2 as published, but train from scratch, which is the less favourable case.

X-GRAM (Chen et al., 2026) looks at hashed memory below 1B parameters. Most entries of a large hashed table receive almost no updates. At 0.73B parameters, Engram's average downstream accuracy rose from 47.2 to 47.7 when its table was doubled, then fell to 45.6 at four times the size; at 1.15B the largest table was best (48.5). This is the regime we work in, and it is the main reason the proposal built on Lngram rather than Engram: in Lngram every token updates one entry per code per order, so entries are trained more evenly.

Memory as experts. Lookup memory is the storage-side counterpart of sparse expert models (Fedus et al., 2022; Dai et al., 2024), which also decouples total parameters from per-token compute, but routes to small FFNs rather than stored vectors. Both Engram and Lngram use an MoE backbone, so their allocation results trade memory against experts. **UltraMem** (Huang et al., 2025b) positions a very sparse memory layer as an alternative to MoE, and takes its input from one layer while adding its output at a later one, so that memory-layer work can overlap with the Transformer layers in between. **UltraMemV2** (Huang et al., 2025a) replaces fixed table vectors with tiny one-unit-wide FFNs per slot, in the spirit of PEER's million tiny experts (He, 2024), and finds a clear gain at the same compute and the same bytes fetched. It also finds that how many entries are active per token matters more than how many exist. Our rank-1 readout (Section 6) is a small version of this idea.

Parameter allocation. Standard decoder-only models such as Llama (Touvron et al., 2023) put roughly two thirds of their non-embedding parame-

ters in FFNs, about 2:1 against attention, and use grouped-query attention (Ainslie et al., 2023) to keep the KV cache small. Scaling studies (Kaplan et al., 2020; Hoffmann et al., 2022) usually fix the architecture's shape and vary overall size. Our own baselines are well off the 2:1 point, which matters for how our results should be read (Section 6). We found little work on how an auxiliary module changes the best split. In a dense model there are no experts to trade against, so FFN width is the only place memory parameters can come from. As far as we can tell, nobody has measured that trade, or reported how much GPU memory a memory-augmented model actually needs when its table is held elsewhere.

3 Problem Statement

Write a model as $\mathcal{M}(\theta_{\text{attn}}, \theta_{\text{ffn}}, \theta_{\text{mem}})$, with θ_{mem} split into the sparse table θ_{tab} and a small dense readout. We use two budgets:

$$N_{\text{tot}} = |\theta_{\text{attn}}| + |\theta_{\text{ffn}}| + |\theta_{\text{mem}}|,$$
$$N_{\text{gpu}} = N_{\text{tot}} - |\theta_{\text{tab}}|.$$

N_{gpu} counts the parameters that must sit in GPU memory if the table is served from host RAM, and is close to the number of parameters active per token. Let $s = |\theta_{\text{tab}}|/N_{\text{tot}}$ be the table share.

Q1 (this report). At fixed N_{tot}, does memory change the best split between attention and FFN? Our *reallocation* removes $\Delta = 1280$ of FFN width in 8 layers and puts the freed 31.5M parameters into attention in all 24 layers (16→24 query heads, 2→4 KV heads). With $\mathcal{L}$ for the memory model and $\mathcal{L}'$ for dense, the interaction is

$$\delta = (\mathcal{L}_{\text{cut}} - \mathcal{L}_{\text{full}}) - (\mathcal{L}'_{\text{cut}} - \mathcal{L}'_{\text{full}}). \qquad (1)$$

Negative values of δ mean the cut is more favourable with memory than without. Our hypothesis was $\delta < 0$, because memory would cover for the lost FFN capacity. We framed this expecting the cut to hurt, but the same sign convention applies when it helps: $\delta < 0$ means it helps more with memory, and $\delta > 0$ means it helps less. A positive δ is what we would expect if memory and wider attention partly do the same work, though it has other possible causes (Section 6). We also stated two hypotheses about which FFN layers matter. Under **H1** (failed lookups), tokens whose lookup fails (high Sink mass) rely on nearby FFNs to rebuild the pattern, so cutting them should hurt those tokens most. Under **H2** (integration), the FFNs just after the memory decode its output, so the cut should hurt low-Sink tokens most.

Q2 (final submission). At fixed N_{tot}, how does quality change as s grows and N_{gpu} shrinks, and which memory design keeps quality flat to the largest s? Equivalently: at fixed N_{gpu}, how much does a host-RAM table help over a dense model of the same GPU size? This is what decides whether memory is worth having in a dense model.

4 Changes from the Proposal

Freed parameters went to attention, not the table. The proposal moved removed FFN width into the table. Our 100M table is already 27% of the budget, and adding 31.5M would take it to about 37%, past the optimum in Lngram v1's allocation study. We sent them to attention instead. Q2 tests the original idea more directly.

Cut layers were chosen by hypothesis. We planned to measure per-layer FFN importance first, but ran out of time and tested two placements instead. They gave the same result. The importance profile is now planned on our own checkpoints.

Not yet done. Block deletion, one of the three core comparisons, has not been run, and there are no downstream benchmarks yet, so the MMLU and PIQA questions are open. Runs used 2B tokens rather than the budgeted ∼8B so that all six would finish in time. The Engram baseline is deferred: our implementation[1] uses a different backbone (768 wide, 12 layers) and needs porting.

Smaller changes. We dropped the UltraMemV2 init, which conflicts with starting the memory branch silent, and the scored readout, since v2's readout already weighs every route against the Sink.

5 Experimental Setup

Backbone. A Llama-style decoder with hidden size 1024, 24 layers, RMSNorm, RoPE ($\theta = 10{,}000$), SwiGLU FFNs and tied embeddings over a 32,768-token byte-level BPE vocabulary (Touvron et al., 2023; Zhang and Sennrich, 2019; Su et al., 2021; Shazeer, 2020). Weights use init std 0.02, with output projections scaled by $1/\sqrt{2L}$. Memory models use $d_{\text{ff}} = 2816$; dense models

[1] https://github.com/MorbiusPlexus/engram

use $d_{\text{ff}} = 4256$ so that both have ∼370M non-embedding parameters (within 0.2%). Table 1 lists the runs; a full parameter breakdown is in Appendix A.1.

Memory. Lngram v2 is vendored unchanged from the official repository (commit `c8a53ca1`): $R = 90$ routes, $d_m = 128$, 2- and 3-gram tables with 4-bit codes, 4 KV readout heads. It is inserted before attention at layers 1 and 11 (counting from 0), following Lngram v1's placement at its layers 2 and 12. We follow the v2 paper's Appendix E recipe: normal table init, zero-initialized output projection and convolution, an initial Sink mass of 0.5, and the approximate surrogate gradient. Tables train at 4× the base learning rate without weight decay, matching the paper's 4:1 ratio between table and other module learning rates. Our wrapper owns the router and computes its logits in fp32. The one deliberate deviation is that document-boundary masking is off in all runs.

Reallocation. The cut lowers d_{ff} by 1280 in 8 layers: 2816→1536 for memory models (a 45% cut) and 4256→2976 for dense (30%). The freed parameters are identical, but a 45% cut may simply cost more than a 30% one, a confound for the interaction. The cut also narrows the FFN, adds query heads and adds KV heads at once, so these runs cannot say which change matters. The insertion layers' FFNs are never cut.

Training. AdamW (Loshchilov and Hutter, 2019) ($\beta = (0.9, 0.95)$, weight decay 0.1), clipping at 1.0, peak LR 6×10^{-4} with 2% warmup and cosine decay to 6×10^{-5}. Batches are 256 sequences of 2048 tokens (524K tokens), for 3,815 steps or 2B tokens. Training is fp16 with a grad scaler, because the 2080 Ti has no bf16. Activation checkpointing is non-reentrant; the reentrant mode silently disables the Lngram surrogate gradient. Data is a 10B-token subset of ClimbMix (Diao et al., 2025), taken from the `OptimalScale/ClimbMix` release on HuggingFace and read in one fixed order shared by all runs, with a separate 20M-token validation slice.

Implementation. The code for all runs in this report[2] is about 2,500 lines of Python with 41 tests.

[2]Ablation code: https://github.com/MorbiusPlexus/project_ablation_code. Engram implementation: https://github.com/MorbiusPlexus/engram. Runs are logged at https://wandb.ai/ad1tya2-iiit-hyderabad/morbius-wave1.

Run	Config	Cut	Attn	Active	GF	Loss
1	Memory	—	16/2	270M	0.64	2.805
2	Mem + cut	near	24/4	270M	0.69	2.791
3	Mem + cut	far	24/4	270M	0.69	2.790
4	Dense	—	16/2	370M	0.84	2.802
5	Dense + cut	near	24/4	370M	0.89	**2.774**
A	Mem + rank-1	—	16/2	270M	0.64	2.807

Table 1: All runs have ∼370M non-embedding parameters and see the same 2B tokens. Cut: near is layers 2–5 and 12–15, far is 6–9 and 16–19. Attn is query/KV heads. Active is non-embedding parameters used per token. GF is our estimate of forward GFLOPs per token (weight matmuls plus attention scores at mean causal context 1024, excluding the output layer); it is not measured. Loss is validation loss on a fixed 20M-token held-out slice. Single seed each.

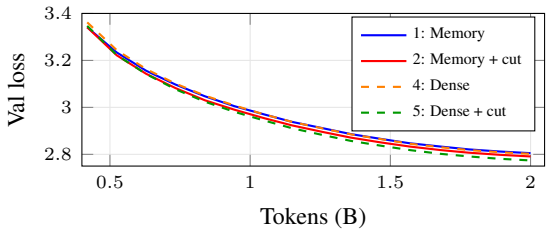


Figure 1: Validation loss from 0.4B tokens onward. Both cut runs start behind (at 0.2B tokens: +0.08 with memory, +0.15 dense) and pull ahead by about 0.5B tokens. The gaps are stable from 1B tokens.

Resume is bit-exact, the data order does not depend on GPU count or micro-batch size, and none of the six runs crashed or diverged.

Hardware, and what it confounds. Runs 1–3 used 4× RTX 2080 Ti with ZeRO-1 (Rajbhandari et al., 2020) and micro-batch 2. Run 4 used 4× RTX 3090, and Runs 5 and A used 2× RTX 3090, all at micro-batch 4 without ZeRO. The global batch is the same, but gradient accumulation, sharding and fp16 reduction order differ. Only comparisons among Runs 1–3, and between Runs 4 and 5 (different GPU counts), stay on one GPU type. Every memory-vs-dense comparison, the rank-1 comparison (A vs. 1) and therefore δ cross hardware, by an amount we have not measured.

6 Results

Table 1 gives final losses and Figure 1 the training curves.

Moving FFN parameters into attention helps. The cut lowers loss in every configuration: by 0.014 and 0.015 with memory (Runs 2 and 3 against Run

1) and by 0.028 without (Run 5 against Run 4). Both cut runs pay a penalty early in training, presumably while the extra heads are still untrained, but are ahead by 0.5B tokens, and the gaps barely move after 1B.

This should not be read as evidence against the usual 2:1 split. Our baselines are far more FFN-heavy than that: 313.8M FFN against 56.6M attention in the dense model (about 5.5:1) and 207.6M against 56.6M with memory (3.7:1), because attention has only two KV heads at hidden size 1024. The cut moves them to 3.2:1 and 2.0:1, so the likelier reading is that our baselines were short on attention. The cut is also not compute-neutral: wider attention raises the cost of attention scores by half, about 6–8% more forward FLOPs per token by our estimate (Table 1), and Run 2 trained about 8% slower than Run 1 on identical hardware. Part of the gain may come from this extra compute.

Where we cut does not matter. Cutting far from the insertion layers (Run 3, 2.790) gives the same result as cutting right after them (Run 2, 2.791), and the curves stay within 0.002 of each other throughout. We also bucketed the per-token loss difference between Runs 2 and 1 by Sink mass on 2M validation tokens. The gap is flat, between -0.012 and -0.022 in every decile, with zero correlation. Neither H1 nor H2 gets any support, but this is weak evidence against them. The differences are tiny and single-seed; both hypotheses concern the early penalty, and our earliest kept checkpoint (step 1000) is past it; and Sink mass turns out to be a poor indicator of memory use (below), so a flat result may only mean the proxy carries little information.

Memory ties dense at equal total size, and loses to the best dense model. At equal total parameters, Run 1 (2.805) and Run 4 (2.802) are within noise, so 105M memory parameters buy about what 106M extra FFN parameters buy. The memory model uses 27% fewer active parameters per token, which by our estimate is about 24% fewer forward FLOPs. We have not measured this: Runs 1 and 4 trained on different GPUs, so their throughputs cannot be compared. The best model overall has no memory, and the cut memory model trails it by 0.017 (2.791 vs. 2.774). Figure 2 puts all six runs on one axis of estimated compute. Memory buys a lower cost per token, not a lower loss. We return to what this does and does not show in Section 7.

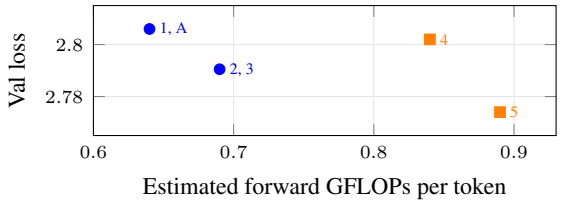


Figure 2: Final loss against estimated compute. Circles are memory models (Runs 1 and A, and Runs 2 and 3, overlap), squares dense. Single seed; the memory and dense points were trained on different GPUs.

The interaction goes the wrong way, but is not yet established. $\delta = (-0.014) - (-0.028) = +0.014$: the cut helps the dense model about twice as much, the opposite of what we expected. We would not call this result reproducible yet. δ combines four runs, so if each final loss carries independent noise σ, δ carries about 2σ, twice the noise of a single run and more than any one pairwise comparison. We have no measured σ. The two halves of δ were also trained on different hardware. Even if the effect is real, there are at least three explanations besides overlap between memory and attention: the memory models lose 45% of the FFN width in the cut layers against 30% for dense; the extra attention compute may be worth more to one model than the other; and query and KV heads change together. The replicates and controls in Section 8 are meant to separate these.

The rank-1 readout adds nothing. Run A adds a 32-dimensional rank-1 path to each table entry at layer 1, paid for by halving the memory at layer 11 (d_m 128→64). It finishes at 2.807, level with Run 1. The path is used but stays small (7% of the residual norm, against 32% for v2). It may be too narrow, its gate is not tied to v2's route weights, or halving layer 11 cost what it gained; we did not separate these.

6.1 How much do the models use their memory?

At the step-2000 checkpoints, we replaced each memory layer's output with zeros and re-evaluated on the full validation slice (Table 2).

Zeroing is a blunt test. The model never saw a zero memory output during training, so part of the loss increase may come from pushing the network off its usual inputs rather than from the information the memory carries. With that caveat, the models clearly depend on their memory. Turning it off costs Run 1 a third of a nat, far more than the 0.003

Run	Normal	L1 off	L11 off	Both off	Out/res.
1	2.970	+0.212	+0.022	+0.328	.33 / .14
2	2.954	+0.073	+0.015	+0.144	.28 / .11

Table 2: Zero-out probe at step 2000. Columns give the loss increase when memory at layer 1, layer 11, or both is zeroed. Out/res. is the mean norm of the memory output relative to the residual stream at layers 1 and 11.

gap between memory and dense. Almost all of this comes from layer 1. Layer 11 contributes little. This is consistent with Lngram v2's finding that addresses are least stable at the deeper insertion layer when the model's representations are still changing, which is always the case when training from scratch, though they did not test our setting directly.

The cut model depends on its memory about half as much (+0.144 against +0.328), and its memory output is smaller relative to the residual stream. One explanation is that the wider attention takes over some of what the memory does, which would also explain why the cut helps dense more. The probe does not show this. It shows that Run 2 relies less on its memory, not what replaced it. Testing the explanation needs a look at the attention itself, for example whether Run 2 has more heads that attend to the previous few tokens than Run 1 does. That analysis is in the plan.

The memory diagnostics point the same way (Appendix A.2). Sink mass drifts back toward its initial 0.5 and routing stays close to uniform, with about 15 of 16 codes in effective use where the papers report about 8. Taken alone these numbers would suggest an unused memory, which the probe contradicts. We therefore do not treat Sink mass as a measure of memory use, which also weakens the Sink-bucketed test above.

7 Is the Memory Worth Having?

The best model we trained has no memory, and memory only ties dense. It is tempting to conclude that the memory is not needed. We think our runs cannot show that, for two reasons.

First, the cut result is probably mostly about attention: our baselines are short on it and the cut adds compute. The memory-specific part is the interaction, which is not yet established, and even if it holds it says memory and attention may overlap, not that memory is redundant.

Second, every run holds N_{tot} fixed, and that is the wrong yardstick for memory. A token reads only about 180 table rows per memory layer (90 routes, two n-gram orders; about 46 KB in fp16), so the table can in principle live in host RAM. A dense FFN cannot be moved off the GPU in the same way, because every token reads all of its weights. Seen this way, Run 1 is a modestly positive result: 27% of its parameters need not occupy GPU memory, at no measurable cost against a dense model of the same total. Against the best dense model the cost is 0.017 (Run 2). Neither the GPU nor the FLOP saving has been measured yet.

What none of our runs can show is whether the table adds anything on top of the 270M parameters the memory model keeps on the GPU. That needs a dense model of $\sim$270M non-embedding parameters with the same 24/4 attention, compared against Run 2. If Run 2 clearly beats it, the table earns its place. If they tie, the table is not contributing at this scale, itself a useful negative result. If the dense model wins, something in our setup is probably hurting the memory.

Offloading also changes which memory design looks better. Lngram's keys come from a hidden state, so the needed rows are known only once the earlier layers have run. Engram's keys come from token IDs and can be prefetched, which is how it hides host-RAM latency. Under an offload framing Engram becomes a serious competitor. Lngram could hide the same latency with delayed injection, computing the key at layer ℓ and adding the result at $\ell + \Delta$ (Huang et al., 2025b).

8 Plan for the Rest of the Project

The final paper will answer Q2: how much of the model can be moved into a host-RAM table before quality drops, and which design lets us move the most? Figure 3 sketches what we are looking for.

No training needed. We start with the six saved checkpoints. We will run lm-eval on HellaSwag, MMLU (cloze), SciQ, WinoGrande, PIQA and ARC-Easy, resampling examples before reading anything into small differences, since MMLU will be near chance at this size. We will repeat Engram's per-task zero-out probe and zero one FFN at a time to get the importance profile. To test whether wider attention takes over local pattern work, we will compare attention patterns across Runs 1, 2, 4 and 5, counting heads that mostly attend to the previous one to three tokens. We will also replace the FLOP estimates in Table 1 with profiled numbers on a sin-

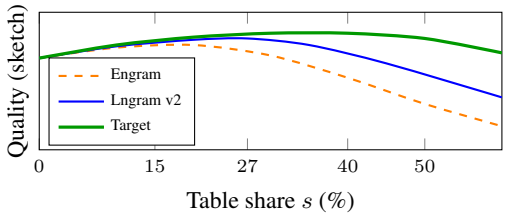


Figure 3: The shape we are testing for. This is a sketch, not data. Total parameters are fixed and FFN width is traded for table size, so moving right means less GPU memory. Published optima sit near 20–25%, and Engram degrades with table size below 1B parameters (Chen et al., 2026). We want a design whose curve stays flat further to the right.

s	R	**Table**	**FFN**	N_{gpu}	**Status**
0%	—	—	282M	370M	Run 5
15%	≈50	55M	221M	314M	planned
27%	90	100M	176M	270M	Run 2
40%	≈133	148M	129M	222M	planned
50%	≈166	185M	92M	185M	planned

Table 3: Planned sweep at $N_{\text{tot}} \approx 370$M with attention fixed at 88M. Dense baselines are planned at 270M and 185M.

gle GPU type. In parallel we will measure batch-1 decode latency and resident GPU memory with the tables on the GPU and in pinned host RAM, for Lngram with $\Delta \in \{0, 1, 2, 4\}$ and for Engram with prefetching. This needs only random weights.

Settling the interaction (week 1). The interaction rests on the weakest evidence, so it comes first. We will rerun Runs 1, 2, 4 and 5 on one GPU type with identical settings (4× RTX 3090, micro-batch 4, no ZeRO), then train a second seed of each. This removes the hardware confound and gives a measured noise level for δ. Two controls go alongside: a memory model whose cut is 30% of FFN width, matching the dense fraction ($\Delta \approx 845$, freeing ∼21M), and a cut without widening, which is not parameter-matched but separates FFN narrowing from attention widening.

The deciding baseline. The dense ∼270M model from Section 7, on the same hardware.

Table-share sweep. Holding N_{tot} and the 24/4 attention fixed, we trade FFN width for routes (Table 3). Comparing each memory point with a dense model of the same N_{gpu} gives two curves of loss against GPU-resident size; the gap between them is what the table is worth.

Baselines from the proposal. We will port Engram to our backbone and train it at 27% and 40% table share. We will also run block deletion at equal total parameters, reproducing Lngram's 23-layer experiment in a dense model, and Run 2 with the freed parameters sent to the table ($R \approx 118$) instead of attention.

Flattening the curve. We do not yet have a design we expect to beat Lngram at high table shares. The most promising option is giving table entries more computation, as UltraMemV2 does (Huang et al., 2025a): a wider per-entry path than our rank-1 attempt, with its gate tied to the route weights. Longer term, a table with a hash-keyed region (prefetchable) and a latent-keyed region (generalizes across wordings) would address the prefetch problem directly.

Timeline and compute. A 2B-token run takes about a day on a 4×2080 Ti node and about 14 hours on the 4×3090 node, and we have four nodes. The roughly sixteen runs above take about five days of wall-clock time. The matched-hardware reruns need the 3090s, so they will be queued first. If compute runs short we will drop the design variants and the 15% sweep point first, then the second seeds of Runs 1 and 4.

- **Week 1:** matched-hardware reruns and second seeds of Runs 1, 2, 4 and 5; matched-fraction and no-widening controls; benchmarks, probes, head analysis, FFN importance and latency on existing checkpoints.
- **Week 1–2:** dense-270M baseline.
- **Week 2:** table-share sweep with matched dense baselines.
- **Week 3:** Engram port and runs; block deletion; table-destination run.
- **Week 4:** design variants as compute allows; LogitLens and CKA.
- **Week 5:** final evaluation and writing.

9 Limitations and Conclusion

Every result here is single-seed, and every memory-vs-dense comparison crosses GPU types. The cut is not compute-matched, cuts a larger fraction of the memory models' FFN, and changes several things at once; our FLOP figures are estimates. We trained for about a quarter of the compute-optimal budget,

have only validation loss, and see routing statistics unlike the papers' (15 effective codes against 8).

With those caveats: shifting parameters from FFN to attention helps our attention-starved baselines, more so without memory; memory ties dense at equal total size with 27% fewer active parameters, though the best model has none; and the models rely heavily on their memory, less so once attention is widened. Our hypothesis that memory would make FFN cuts cheaper did not hold. What remains open, and what the rest of the project will answer, is whether a dense model can keep a large part of its parameters in a host-RAM table without losing quality.

References

Joshua Ainslie, James Lee-Thorp, Michiel de Jong, et al. 2023. GQA: Training generalized multi-query transformer models from multi-head checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 4895–4901.

Yilong Chen, Yanxi Xie, Zitian Gao, et al. 2026. Beyond n-gram: Data-aware X-GRAM extraction for efficient embedding parameter scaling. *arXiv preprint arXiv:2604.21724*.

Xin Cheng, Rui Tian, Wangding Zeng, et al. 2026. Conditional memory via scalable lookup: A new axis of sparsity for large language models. *arXiv preprint arXiv:2601.07372*.

Damai Dai, Chengqi Deng, Chenggang Zhao, et al. 2024. DeepSeekMoE: Towards ultimate expert specialization in mixture-of-experts language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1280–1297.

Shizhe Diao, Yu Yang, Yonggan Fu, et al. 2025. Nemotron-CLIMB: CLustering-based iterative data mixture bootstrapping for language model pre-training. *arXiv preprint arXiv:2504.13161*.

William Fedus, Jeff Dean, and Barret Zoph. 2022. A review of sparse expert models in deep learning. *arXiv preprint arXiv:2209.01667*.

Xu Owen He. 2024. Mixture of a million experts. *arXiv preprint arXiv:2407.04153*.

Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, et al. 2022. Training compute-optimal large language models. In *Advances in Neural Information Processing Systems*.

Zihao Huang, Yu Bao, Qiyang Min, et al. 2025a. UltraMemV2: Memory networks scaling to 120B parameters with superior long-context learning. *arXiv preprint arXiv:2508.18756*.

Zihao Huang, Qiyang Min, Hongzhi Huang, et al. 2025b. Ultra-sparse memory network. In *International Conference on Learning Representations*. ArXiv:2411.12364.

Jared Kaplan, Sam McCandlish, Tom Henighan, et al. 2020. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*.

Ilya Loshchilov and Frank Hutter. 2019. Decoupled weight decay regularization. In *International Conference on Learning Representations*.

nostalgebraist. 2020. Interpreting GPT: the logit lens. LessWrong blog post.

Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. 2020. ZeRO: Memory optimizations toward training trillion parameter models. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC20)*.

Noam Shazeer. 2020. GLU variants improve transformer. *arXiv preprint arXiv:2002.05202*.

Jianlin Su, Yu Lu, Shengfeng Pan, et al. 2021. RoFormer: Enhanced transformer with rotary position embedding. *arXiv preprint arXiv:2104.09864*.

Hugo Touvron, Louis Martin, Kevin Stone, et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*.

Ashish Vaswani, Noam Shazeer, Niki Parmar, et al. 2017. Attention is all you need. In *Advances in Neural Information Processing Systems*.

Biao Zhang and Rico Sennrich. 2019. Root mean square layer normalization. In *Advances in Neural Information Processing Systems*.

Yunao Zheng, Bin Wen, Xiaojie Wang, et al. 2026a. Lngram v2: Latent n-gram memory with interpretable discrete representations. *arXiv preprint arXiv:2609.03426*.

Yunao Zheng, Guoyang Xia, Xiaojie Wang, and Lei Ren. 2026b. Lngram: N-gram conditional memory in latent space. *arXiv preprint arXiv:2605.24869*.

A Supplementary Details

A.1 Parameter breakdown

	Mem	Mem cut	Dense	Dense cut
Attention	56.6M	88.1M	56.6M	88.1M
FFN	207.6M	176.2M	313.8M	282.3M
Tables	100.3M	100.3M	—	—
Readout	5.2M	5.2M	—	—
Non-embedding	369.8M	369.8M	370.5M	370.5M
Active/token	269.5M	269.5M	370.5M	370.5M

Table 4: Parameters by component. All models also have a 33.6M tied embedding.

A.2 Memory diagnostics (Run 1)

Step	Sink L1	Sink L11	Codes L1	Codes L11	Table grad
200	.461	.434	14.6	13.0	.011
1000	.453	.453	15.0	14.6	.002
2200	.463	.451	14.9	14.6	.001
3815	.470	.451	14.8	—	—

Table 5: Sink mass (starts at 0.5) and effective codes per route (of 16). No dead routes in any run.

A.3 Hardware and stability

Runs 1–3 used 4× RTX 2080 Ti with ZeRO-1 at micro-batch 2 (about 21–23K tokens/s). Run 4 used 4× RTX 3090 at micro-batch 4 (42K tokens/s); Runs 5 and A used 2× 3090 (about 20K tokens/s). Global batch is the same everywhere. Before launch, each configuration was checked for peak GPU memory with one full training step. Run 5 had a brief loss spike during warmup (train loss 6.49 to 6.96 between steps 60 and 80, gradient norm 14.5) with the fp16 scale unchanged, and had overtaken Run 4 by step 1000.